Leo Gadian – Teledidattica – 2087727

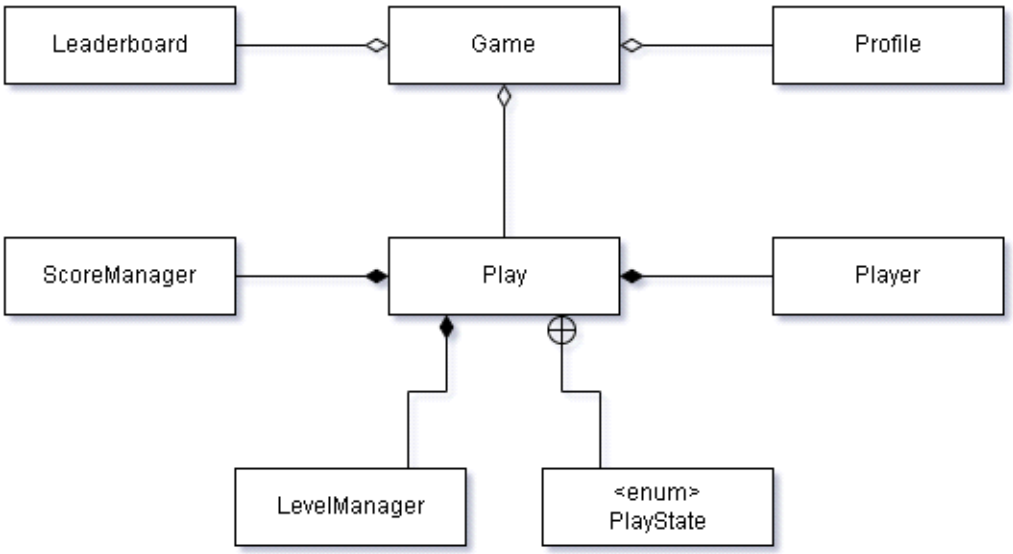
JImpossibleMission

Model

Il model è la parte del progetto dove racchiude tutta la logica e le informazioni del gioco.

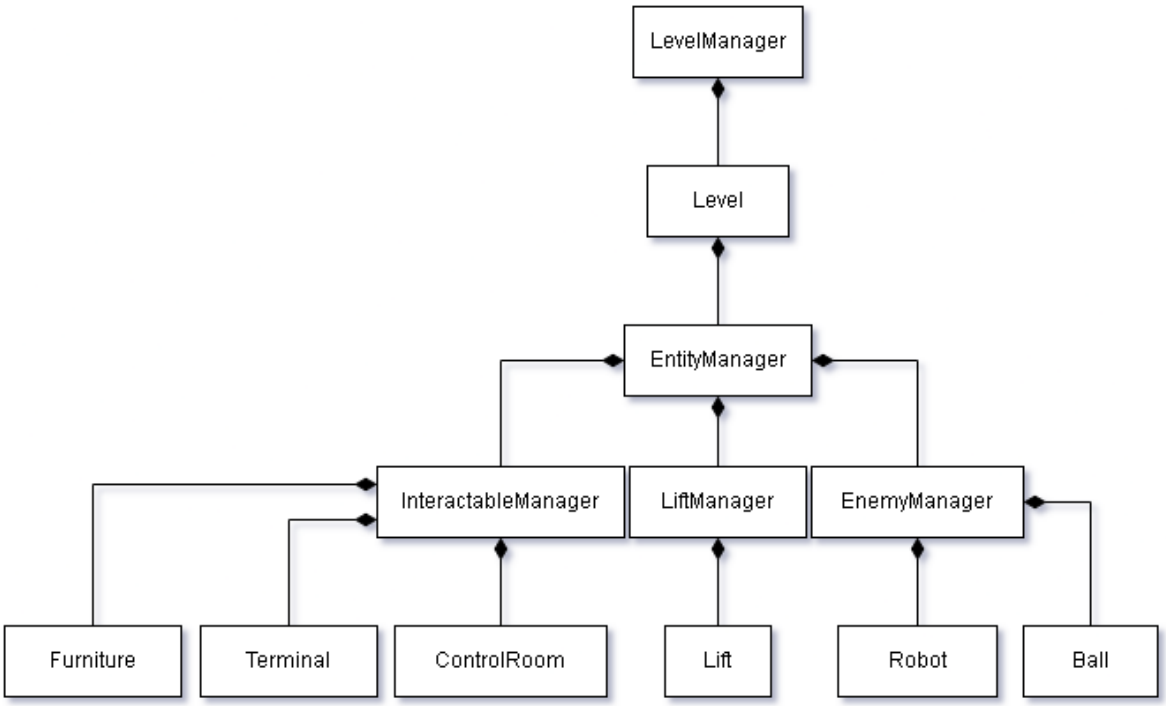
Iniziando con la classe **Game** dove inizia tutto, gestisce gli 8 profili e la classifica (caricandoli, salvandoli e modificandoli) e inizializza/ferma il **Play** che sarebbe la partita in sé.

- Il **Profile** salva il nickname, il nome dell'immagine avatar, le giocate, le vittorie e le perdite del profilo insieme il totale di punteggio che ha accumulato fino adesso (usato per calcolare il suo livello che aumenta ogni 5000 punti).
- Il **Leaderboard** rappresenta la classifica salvato fino adesso, utilizzando un'altro tipo di oggetto chiamato **RecordSave** (che conserva il nome del profilo che ha giocato nel momento, salvando solo i primi cinque caratteri, e il punteggio guadagnato nella partita) che rappresenta i record e li salva in una lista. Qui puoi aggiungere un record, salvare o resettare la classifica.
- Il **Play** gestisce l'aggiornamento, il punteggio (**ScoreManager**, che prende i dati dal **Play** stesso per calcolare il punteggio alla fine di una partita) e lo stato della partita (**PlayState**), creando il **Player** e la gestione dei livelli (**LevelManager**).



Dal **LevelManager** si crea una gerarchia per i livelli e le sue entità dove:

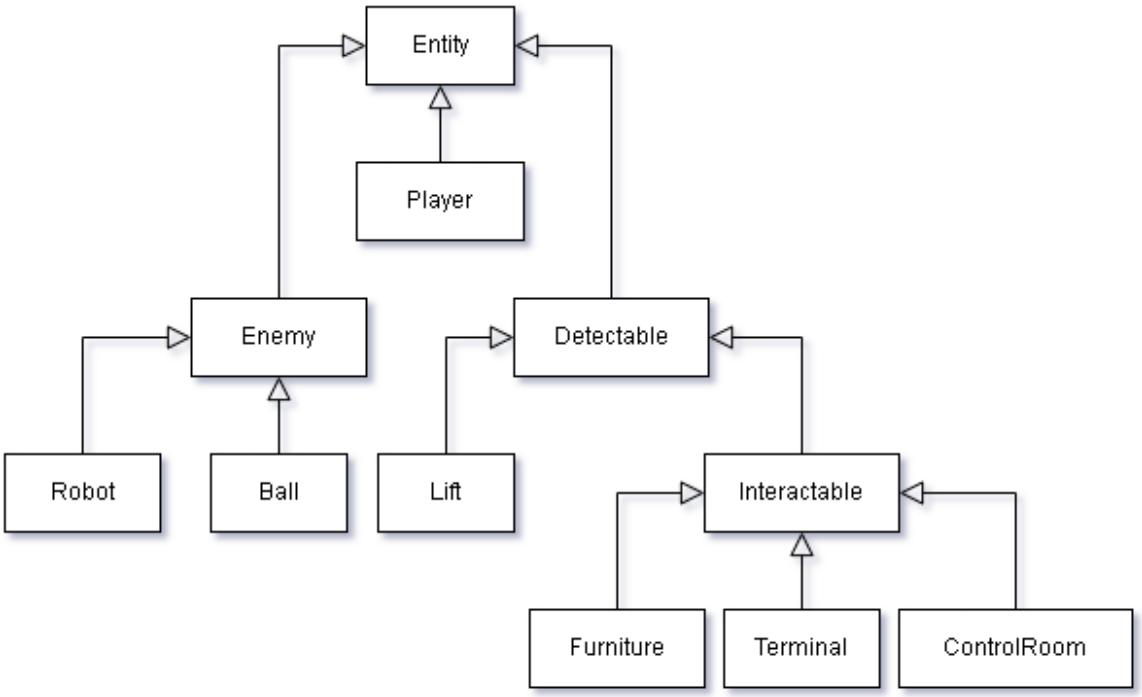
- Il **LevelManager** carica i **Level**, leggendo i file txt dalla cartella rooms_data dentro a resource, e li separa nelle loro sottoclassi in una lista di **Room** (stanze dove si cerca i pass e dove si trovano i nemici) e in una lista di **Elevator** (stanze usate come passaggio tra i Room attraverso un ascensore dove è possibile muoverlo su e giù).
- **Level** (come anche le sue sottoclassi, **Room** e **Elevator**) contiene le entità all'interno e gestita con **EntityManager**.
- **EntityManager** carica tutte le entità, usando lo stesso nome del file usato per i **Level** ma in una cartella diversa (entities_data), e lì distribuisce in tre diversi manager dipendentemente dal tipo di entità: **EnemyManager** che gestisce le entità **Enemy** (**Robot** e **Ball**), **LiftManager** che gestisce i **Lift**, **InteractableManager** che gestisce le entità **Interactable** (**Furniture**, **Terminal** e **ControlRoom**).



A sua volta pure le entità hanno una loro gerarchia che si suddivide in tipi che, visto prima, sono distribuite nei loro manager:

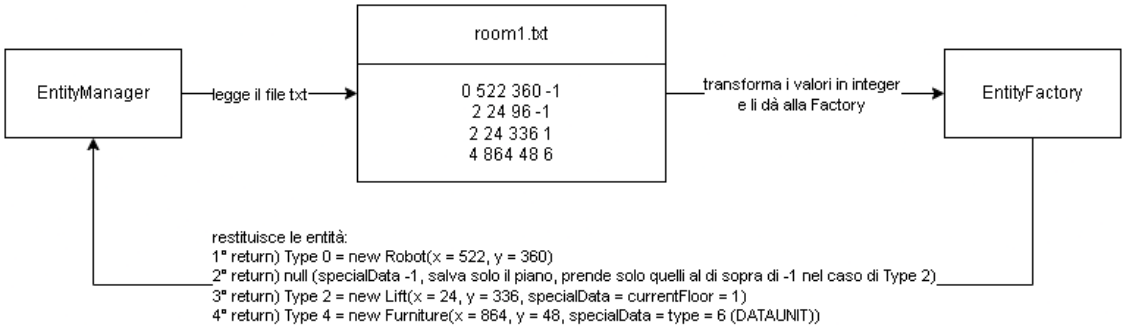
- **Entity** è il superclasse generale delle entità dove contiene la loro hitbox (un rettangolo che rappresenta l'entità, utilizzata per le collisioni), la loro posizione d'origine in cui viene piazzata inizialmente nel **Level** (usata quando si deve resettare il Level) e le sue misure (width e height). L'unica sottoclasse concreta che si estende direttamente in questa classe è il **Player** visto che non appartiene a nessun'altro tipo di entità.
- **Enemy** estende a **Entity** e insieme all'hitbox contiene anche un altro rettangolo che rappresenta la loro atkbox (utilizzata per contatti con i nemici o i loro attacchi, eliminando l'**Entity** bersaglio). I nemici sono: **Ball** (una palla volante che insegue il giocatore costantemente) e **Robot** (un nemico robot terrestre che ha atteggiamenti casuali, spiegato poi dopo).
- **Detectable** estende a **Entity** e insieme all'hitbox contiene anche un'altro rettangolo che rappresenta la loro dtcbox (a differenza dell'atkbox, questo viene solo utilizzata per leggere se l'**Entity** bersaglio è in mezzo o all'interno del suo perimentro). L'unico **Detectable** che non è anche un **Interactable** è **Lift** (che usa dtcbox solo per leggere se il **Player** è sopra oppure se c'è un **Entity** che è in mezzo alla sua via mentre si muove).

- **Interactable** estende a **Detectable** che oltre ad avere il dtcbox ha anche un metodo astratto che servirà per l'interazione con questo oggetto, come i **Furniture** (che aggiornano i loro percentuale della ricerca), i **Terminal** (che ferma il gioco temporaneamente per accedere alla sua schermata) e il **ControlRoom** (che controlla prima se il **Player** ha il pass completo, se sì allora si passa alla vittoria della partita).

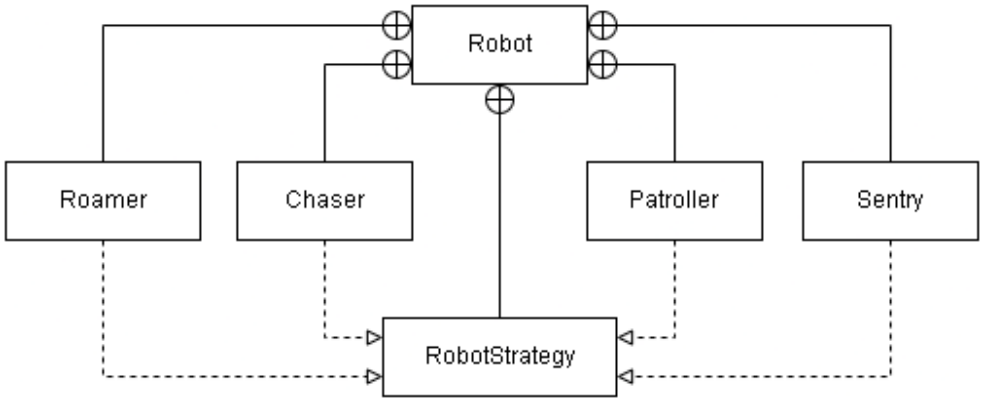


Nel model è stato applicato due pattern:

- **Simple Factory Pattern** per la creazione delle entità, che dando la posizione per piazzarla (x, y) e il tipo (con alcuni casi fa anche utilizzo di un dato in più chiamato specialData), letti da un file txt dal **EntityManager**, verrà passato attraverso ad uno switch che leggerà il tipo di entità deve creare per poi restituirla in output.



- **Strategy Pattern** a solo un nemico che sarebbe **Robot**, utilizzando un'interfaccia **RobotStrategy** con al suo interno un metodo execute() che esegue l'azione del nemico. Con questo possiamo creare diversi comportamenti e attacchi che alla sua creazione potrà scegliere casualmente (utilizzando Random per scegliere una strategia a caso salvato in un array), dando ad ogni partita nuovi modi per approcciare questo nemico.



Gli stream sono stati utilizzati maggiormente per la creazione di alcune liste temporanee o semplici da creare oppure per fare dei determinati azioni a certi oggetti, raggruppati in una lista, che rispettano una certa condizione.
Nel caso del model uno degli stream viene utilizzato per raccogliere certi tipi di **Entity**, muovere o fermare i **Lift** a seconda del loro stato oppure individuare se il **Player** è vicino a un **Detectable**.

```java
List<Robot> robots = entities.stream()
        .filter(entity -> entity instanceof Robot)
        .map(entity -> (Robot)entity)
        .collect(Collectors.toList());
```

Filtra la lista di entità prendendo solo i Robot

Fa il casting trasformando ogni elemento da Entity a Robot

Racchiude in una lista

```java
lifts.stream()
.filter(l -> l.getStatus() != 0)
.forEach(l -> {
    if (l.isNear(p))
        l.push(p);
    ball.ifPresent(b -> {
        if (l.isInTheWay(b))
            l.push(b);
    });
    l.update();
});
```

Prende solo i Lift che hanno lo stato in movimento

Da lì per ogni Lift che si muovono, oltre ad aggiornare il suo movimento, spinge anche il Player (o il Ball se è presente) verso la direzione in cui sta andando se è di intralcio

```java
furnitures.stream()
 .filter(f -> f.isNear(player) && f.isActive())
 .findFirst()
 .map(f -> {
     currentInteractable = f;
     return true;
 }).orElse(false);
```

Filtra solo il Furniture che è vicino al giocatore e che è attivo

Prende il primo presente

Oltre ad impostare come l'interaggibile attuale ritorna come true se è presente, se no ritorna false

View

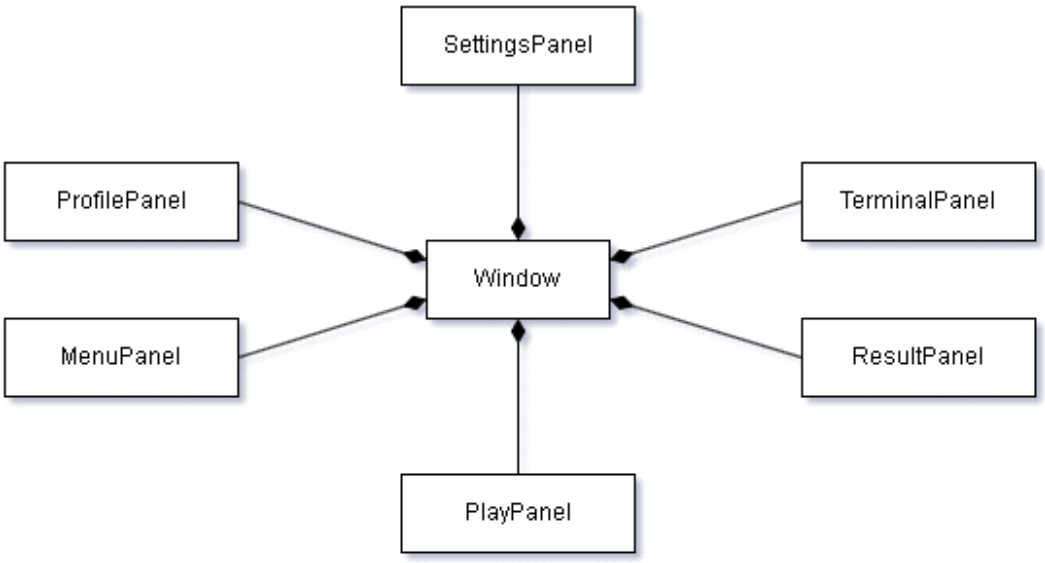
La view è la parte output del progetto, prendendo informazioni dal model per poi rappresentarlo in modo auditivo e visivo.
E' principalmente formato da un JFrame chiamato Window che può cambiare Panel tra quelli che ha inizializzato, utilizzando un'altra classe chiamato Navigator per navigare tra di loro.

Tra i **Panel** di **Window** abbiamo:

- **MenuPanel**, che rappresenta il menu principale che potremo avere accesso alla partita (**PlayPanel**), ai profili (**ProfilePanel**) e alle impostazioni (**SettingsPanel**).
- **PlayPanel** cerca di rappresentare il gioco in modo auditivo e visivo per l'utente, prendendo **Play** per leggere i dati al suo interno (facendo attenzione di non modificarlo in nessun modo) e che verranno utilizzati dalle classi **RenderPlay (**per disegnare i sprite per i livelli e le azioni delle entità, prendendoli dal **SpriteCollections**) e **AudioPlay** (utilizzata per azionare gli audio adatti all'occasione).
- **TerminalPanel** è utilizzata quando il giocatore interagisce con un **Terminal**, dal **PlayPanel** cambierà in questa schermata che avrà accesso ad alcuni pulsanti per

cui il ripristino dei **Lift** nella loro posizione originale oppure disabilitazione di tutti i nemici nella stanza in modo temporaneo.

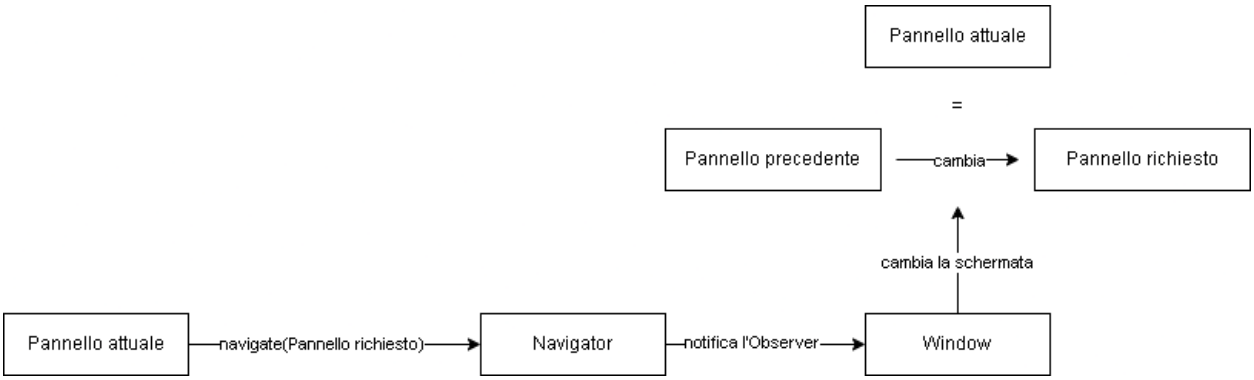
- **ResultPanel** è la schermata dopo aver completato una partita dove ti mostra i punteggi accumulati nella partita e la classifica accumulata fino adesso.
- **ProfilePanel** è la schermata dove puoi personalizzare fino ad otto profili, cambiandogli il nome e l'avatar mentre ti mostra anche quante volte hai giocato, vinto, perso e il tuo livello attuale (mostrando la quantità di punteggi che ha accumulato nel profile e a quanto ti serve per il prossimo livello).
- **SettingsPanel** è la schermata per le impostazioni che viene utilizzata principalmente per disabilitare gli audio (tra la musica del menu e l'audio generale del gioco) e mostrare/resettare l'attuale classifica accumulata fino adesso.



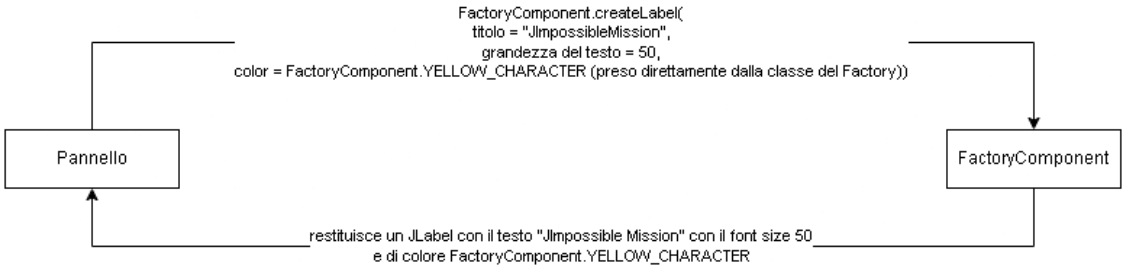
Nella view è stato applicato tre pattern:

- **Observer Pattern**, utilizzato principalmente su **Navigator** (che viene condivisa a tutti i **Panel**) con **Window** come **Observer**. Lo notifica il nome del pannello dato al

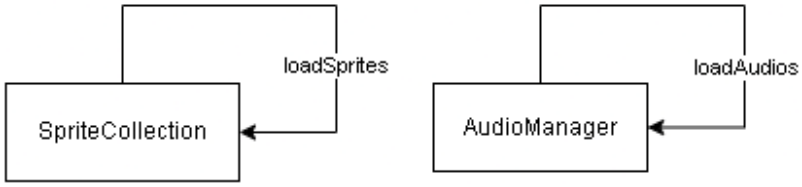
metodo navigate così da lì **Window** può mostrare la schermata richiesta.



- **Simple Factory Pattern**, come nel model viene utilizzato per la creazione di oggetti ma stavolta per i componenti che verranno utilizzati all'interno dei **Panel** come i label, font, colori e pure un JButton personalizzato chiamato **InteractiveButton** (che reagisce con il mouse cambiando la grandezza dei bordi).



- **Singleton Pattern**, utilizzata soprattutto per il caricamento degli asset (**SpriteCollection** e **AudioManager**) per impedire di caricare ogni volta gli sprite o degli audio singolari (che sono utilizzati solo una volta in una partita) quando vengono chiamati, facendolo solo in una volta quando viene chiamata la classe per la prima volta.



Invece gli stream nel caso della view, alcuni di questi viene utilizzato per modificare un gruppo di oggetti come per il gruppo di label ID e punteggi per la classifica per resettare al loro stato default per solo quelli modificati, creazione di una mappa dato un array di nomi degli audio singolari oppure rimostrare/riabilitare certi label o bottoni.

```
ids.stream()
.filter(r -> r.getText() != DEFAULT_ID)
.forEach(r -> r.setText(DEFAULT_ID));
points.stream()
.filter(r -> r.getText() != DEFAULT_POINTS)
.forEach(r -> r.setText(DEFAULT_POINTS));
```

Filtra solo quelli modificati controllando se il testo è diverso da quello di default

Li setta al testo default

```
singularAudios = Arrays.stream(SINGULAR_AUDIONAMES)
        .collect(Collectors.toMap(audioNome -> audioNome, audioNome -> getClip(audioNome)));
```

Rende uno stream l'array con i nomi degli audio singolari

Lo racchiude in una Map con il nome dell'audio come key e la sua Clip come value

```
buttons.stream()
.filter(b -> !b.isVisible())
.forEach(b -> b.setVisible(true));
notif.stream()
.filter(n -> n.isVisible())
.forEach(n -> n.setVisible(false));
```

Filtra la lista (con i bottono solo quelli nascosti mentre con i label solo quelli visibili)

Setta la visibilità (per i bottono li mostra, mentre con i label li nasconde)

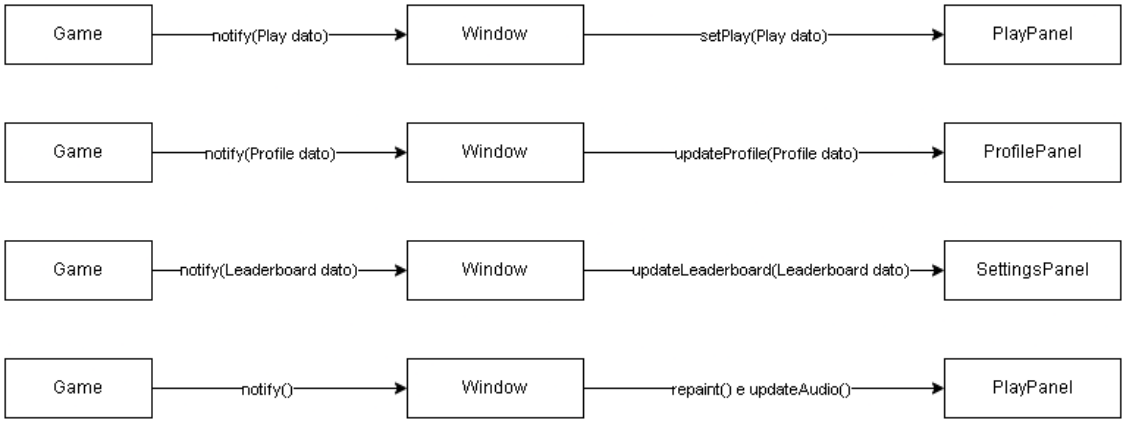
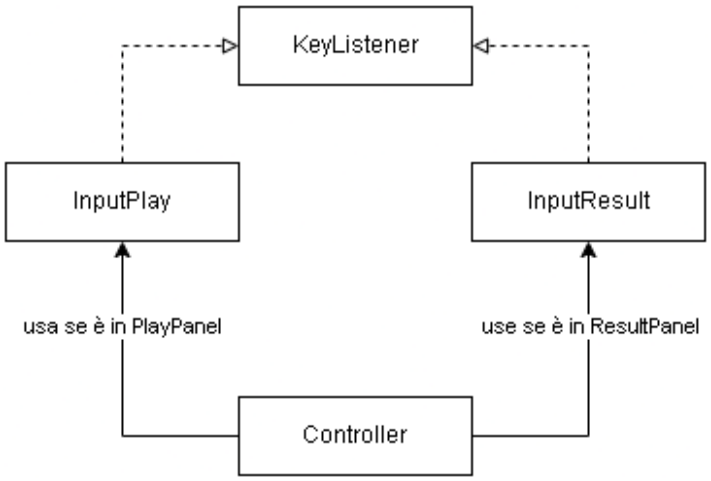
Controller

Il controller viene utilizzato come collante e azionatore tra model e view, dando i componenti della view la loro funzione e inizializzando anche i comandi keyListeners e gli observer per il gioco.

Nel controller viene applicato due pattern:

- **Observer Pattern** per la comunicazione tra model e view, ogni volta che il model (**Game** è l'**Observable**) fa un'azione verrà notificato alla view (**Window** è l'**Observer**) che dipendentemente dall'oggetto inviato farà certe azioni:
  1) Se viene notificato un **Play** allora verrà settata al **PlayPanel** per accedere ai suoi dati per il rendering.
  2) Se viene notificato un **Profile** allora cambierà profilo mostrato nel **ProfilePanel**.
  3) Se viene notificato il **PlayState** (il risultato della partita) imposterà il **ResultPanel** (nel caso di WIN/LOSE) modificando i suoi label e cambiando la schermata in questa da **PlayPannel**, oppure ritornerà nella schermata **MenuPanel** (nel caso di FORCEDSTOP).
  4) Se invece viene notificata con nessun oggetto significa che il **Play** è stato aggiornato e quindi bisogna aggiornare anche il **PlayPanel** con un repaint e un

updateAudio.



- **Strategy Pattern**, che durante alla creazione del controller viene anche inizializzato due tipi di **KeyListeners** che verranno letti dipendentemente da quale schermata si trova l'utente in questo momento. Se l'utente è nel **PlayPanel** il **KeyListener** sarà quello dell'**InputPlay**, che controllerà le azioni del **Player** e le sue interazioni nel gioco, mentre se è in **ResultPanel** avrà solo i comandi per poter ricominciare a giocare (R) oppure tornare al menu (ESC).